

[Home](#) > [Blog](#) > [PostgreSQL](#) > [PostgreSQL 19 Beta: Every New Feature That Matters to DBAs](#)

PostgreSQL

PostgreSQL 19 Beta: Every New Feature That Matters to DBAs

PostgreSQL 19 Beta 1 (June 4, 2026) brings parallel autovacuum, the native REPACK command for online table rebuilds, 2x faster inserts under foreign-key load, online logical replication without a restart, WAIT FOR LSN for read-your-writes consistency, and default changes (JIT off, lz4 TOAST, RADIUS removed). A DBA-focused walkthrough of what changed and what to test before GA.

 JusDB Team  June 15, 2026  Updated June 20, 2026  14 min read

[Performance](#)[PostgreSQL](#)[Migration](#)[Open Source](#)[Replication](#)

PostgreSQL 19 Beta 1 landed on **June 4, 2026**, and it is the most consequential release for operations teams since the asynchronous I/O subsystem debuted in PostgreSQL 18. With over 200 changes carried forward, PostgreSQL 19 is less about flashy syntax and more about the things DBAs fight with every week: vacuum that finally goes parallel, online table rewrites without a maintenance window, insert paths that are twice as fast under foreign-key load, and the ability to flip logical replication on without a restart. This guide walks through every feature that matters to people who run PostgreSQL in production — what changed, why it changed, and what you should actually test before general availability lands in September/October 2026.

BETA, NOT PRODUCTIONWe use cookies for analytics. [Privacy](#)[Reject](#)[Accept](#)

PostgreSQL 19 Beta 1 is for testing only. Behaviors, catalog definitions, and APIs can still change before GA, and there is no upgrade path guarantee between betas. Test it against a copy of your real workload — not toy data — and report anything surprising to the Open Items list. Do not point production traffic at it.

TL;DR — WHAT'S WORTH YOUR ATTENTION

- **Parallel autovacuum** — autovacuum workers can now parallelize index vacuuming via `autovacuum_max_parallel_workers``, plus a new scoring system to prioritize the tables that need it most.
- **REPACK** — a built-in, optionally non-blocking (`CONCURRENTLY`` ) replacement for `VACUUM FULL`` and `pg_repack`` to rebuild bloated tables without an exclusive lock.
- **2x faster inserts with foreign keys** and improved `LISTEN`` / `NOTIFY`` scalability.
- **Online logical replication** — enable logical replication while `wal_level = replica``, without restarting the server.
- **WAIT FOR LSN** — a first-class command for read-your-writes consistency against replicas.
- **Default changes that will surprise you:** JIT is now *off* by default and TOAST compression defaults to `lz4``.
- **RADIUS authentication is removed**, and `md5`` auth keeps marching toward deprecation.

Table of Contents

Vacuum Finally Goes Parallel

REPACK: Online Table Rebuilds Without the Extension

The Insert and Notify Performance Wins

Plan Stability: `pg_plan_advice` and `pg_stash_advice`

Asynchronous I/O Grows Up

Operational Game-Changers for HA Teams

Online Logical Replication (No Restart)

WAIT FOR LSN: Read-Your-Writes Done Right

Online Checksums

Partition Management Lands In-Core

SQL Features Developers Will Reach For

Security and Authentication Changes

Observability: More Signals, Less Guessing

Default Changes That Will Bite the Unprepared

Query Federation: `postgres_fdw` Gets Smarter

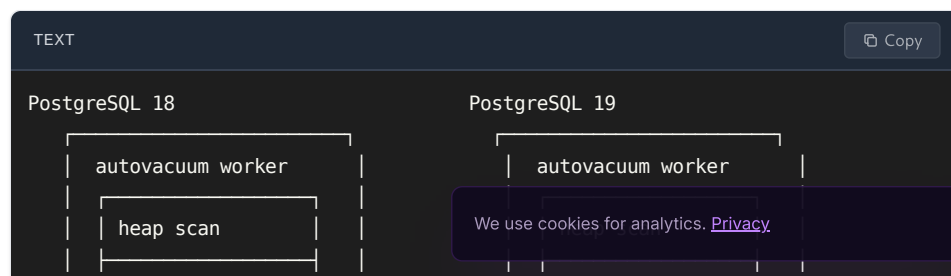
The Upgrade Checklist

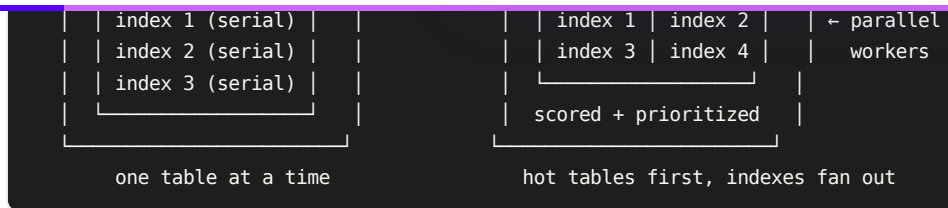
Should You Care Yet?

Vacuum Finally Goes Parallel

For years, the single most common "why is my big table slow to maintain" answer was that a single autovacuum worker chews through one table's indexes serially. PostgreSQL 19 changes that. Autovacuum can now run **parallel workers**, controlled by the new `autovacuum_max_parallel_workers`` setting, so the index-vacuuming phase on large tables is spread across multiple processes the same way a manual `VACUUM (PARALLEL n)`` already could.

Just as important is a new **autovacuum scoring system** that prioritizes which tables to vacuum first, instead of the older mostly-FIFO behavior. On a busy cluster with hundreds of tables and skewed write patterns, this means the table that is actually generating the most dead tuples and bloat gets attention before the launcher wanders off to a cold table that happened to cross its threshold.





There is also a subtler win: a new vacuum strategy that can mark pages as all-visible *while they are still being queried*, which keeps the visibility map fresher and helps index-only scans stay fast on actively-read tables.

WHAT TO TEST

Take your largest, most-indexed table and compare autovacuum duration on 18 vs 19 with `autovacuum_max_parallel_workers` set above zero. Watch `pg_stat_progress_vacuum` — it now exposes a `started_by` column (who kicked it off) and a `mode` column (the vacuum operating mode), so you can finally tell a manual vacuum apart from an autovacuum-triggered one at a glance.

REPACK: Online Table Rebuilds Without the Extension

Bloat reclamation has always been the awkward part of running PostgreSQL. `VACUUM FULL` reclaims space but takes an `ACCESS EXCLUSIVE` lock — a hard outage for the table. The community answer has been the third-party `pg_repack` extension, which is excellent but is something you have to install, version-match, and trust outside of core.

PostgreSQL 19 introduces a native `REPACK` command, including a non-blocking `CONCURRENTLY` option that rebuilds a table to remove bloat while reads and writes keep flowing.

```
SQL Copy

-- Rebuild a bloated table online, no exclusive lock held for the duration

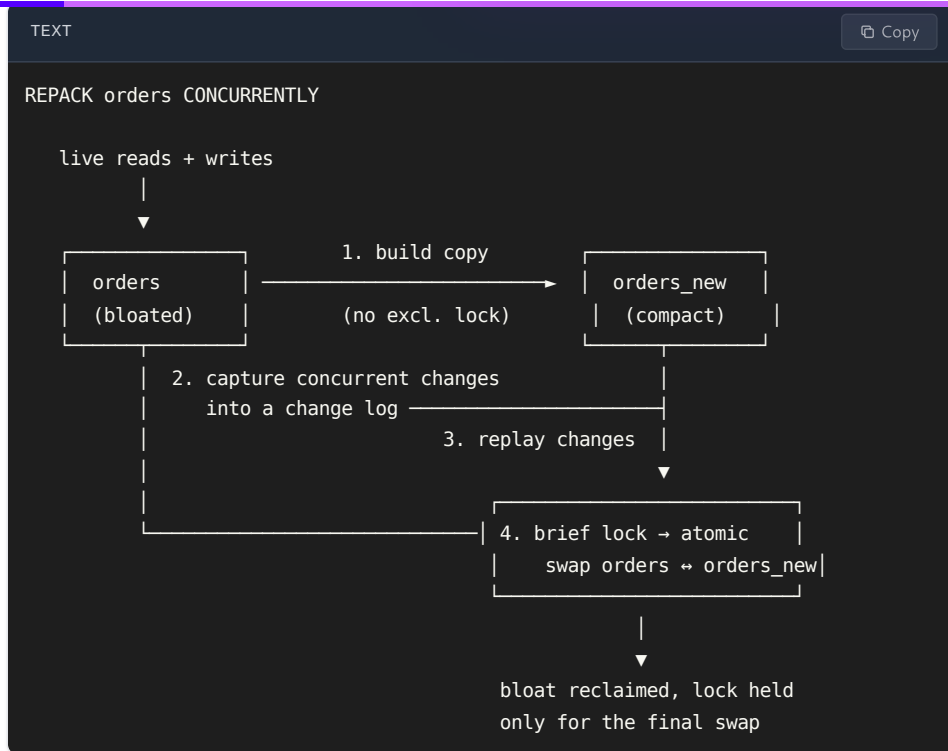
REPACK orders CONCURRENTLY;

-- Rebuild and re-cluster by an index's order

REPACK orders USING INDEX orders_created_at_idx CONCURRENTLY;
```

Under the hood, `REPACK ... CONCURRENTLY` builds a fresh, bloat-free copy of the table while live traffic keeps hitting the original, captures the changes that happen during the rebuild, replays them onto the new copy, and finally swaps the two under a brief lock — the same pattern `pg_repack` pioneered, now baked into core.

We use cookies for analytics. [Privacy](#)



For teams who have been carrying `pg_repack` as a dependency purely to avoid `VACUUM FULL` downtime, this is a meaningful simplification: one fewer extension to manage across every cluster, and a feature that the core project now tests and supports directly.

STILL VERIFY ON YOUR DATA

`CONCURRENTLY` trades the exclusive lock for extra I/O and disk — it builds a fresh copy of the table. Make sure you have headroom on the data volume before running it against a multi-hundred-GB table, and benchmark the runtime on a restore of production so you size the maintenance budget correctly.

The Insert and Notify Performance Wins

Two changes here will show up in real workloads without you touching a single setting:

- **~2x better performance on inserts when foreign key checks are present.** Anyone bulk-loading into normalized schemas — order lines referencing orders, events referencing entities — has paid the FK-check tax on every row. PostgreSQL 19 roughly halves it.
- **Improved `LISTEN` / `NOTIFY` scalability for multi-channel workloads.** Applications using PostgreSQL as a lightweight pub/sub bus across many channels hit contention in older versions; 19 relieves a lot of it.

The planner got attention too. PostgreSQL 19 adds anti-join optimizations, broader use of incremental sorts, a new `enable_eager_aggregate` option to push aggregation lower in the plan for faster row processing, and a quiet-but-nice simplification: `IS DISTINCT FROM` and `IS NOT DISTINCT FROM` collapse to plain `=` and `<>` when the planner

can prove the inputs are not nullable, which unlocks index usage that the verbose form previously blocked.

Plan Stability: `pg_plan_advice` and `pg_stash_advice`

One of the longest-standing operational complaints about PostgreSQL is the lack of a clean, built-in way to pin a plan that suddenly regressed. PostgreSQL 19 ships two new extensions toward that goal:

- `pg_plan_advice` — lets you capture and influence planner decisions to stabilize plans that the cost model gets wrong.
- `pg_stash_advice` — applies that advice automatically, keyed off query identifiers, so a known-good plan sticks without manual intervention on every execution.

This is not Oracle-style hints embedded in SQL, but it is the most serious in-core answer yet to "this query was fine yesterday and the planner flipped to a bad join order overnight."

Asynchronous I/O Grows Up

PostgreSQL 18 introduced the AIO subsystem. PostgreSQL 19 makes it operationally sane. With `io_method = worker`, the number of I/O workers now **scales automatically** between two new bounds:

```
TEXT Copy

# postgresql.conf

io_method = worker

io_min_workers = 1      # floor — always keep this many I/O workers

io_max_workers = 8      # ceiling — scale up to this under load
```

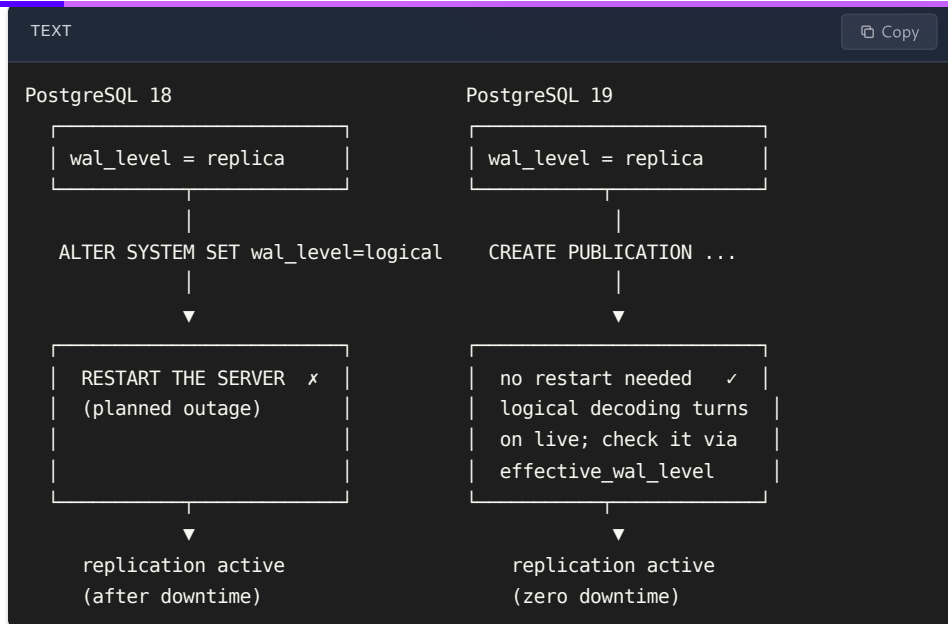
Instead of guessing a static worker count, the server expands and contracts the pool with demand. And when you profile a query, `EXPLAIN ANALYZE` now reports AIO statistics through the `IO` option, so you can actually see how much of a plan's time was spent waiting on asynchronous reads.

Operational Game-Changers for HA Teams

Online Logical Replication (No Restart)

Historically, enabling logical replication meant setting `wal_level = logical` and **restarting the server** — a planned outage on the primary. PostgreSQL 19 lets you enable logical replication online while `wal_level` is still set to `replica`, no restart required. A new read-only `effective_wal_level` parameter reports the WAL level actually in effect, so you can confirm what the server is really doing versus what the config file says.

We use cookies for analytics. [Privacy](#)



Paired with that:

- **Sequence replication** is now supported, which dramatically simplifies online major-version upgrades via logical replication — sequences no longer drift and need manual fix-up at cutover.
- `CREATE PUBLICATION ... EXCEPT`` — publish all tables except a named set, instead of enumerating everything.
- `CREATE SUBSCRIPTION ... SERVER`` — define subscriptions against a foreign server, so connection credentials live in one managed place instead of being baked into each subscription string.

WHY THIS MATTERS FOR MIGRATIONS

Sequence replication plus online enablement removes two of the biggest sharp edges in zero-downtime major upgrades. If you run logical-replication-based upgrades for clients, PostgreSQL 19 makes the cutover meaningfully less risky.

WAIT FOR LSN: Read-Your-Writes Done Right

Read replicas are great until a user writes a row on the primary, immediately reads from a replica, and the replica hasn't caught up yet. Every shop has hacked around this with sticky sessions or application-side polling. PostgreSQL 19 gives you a real command:

SQL Copy

```

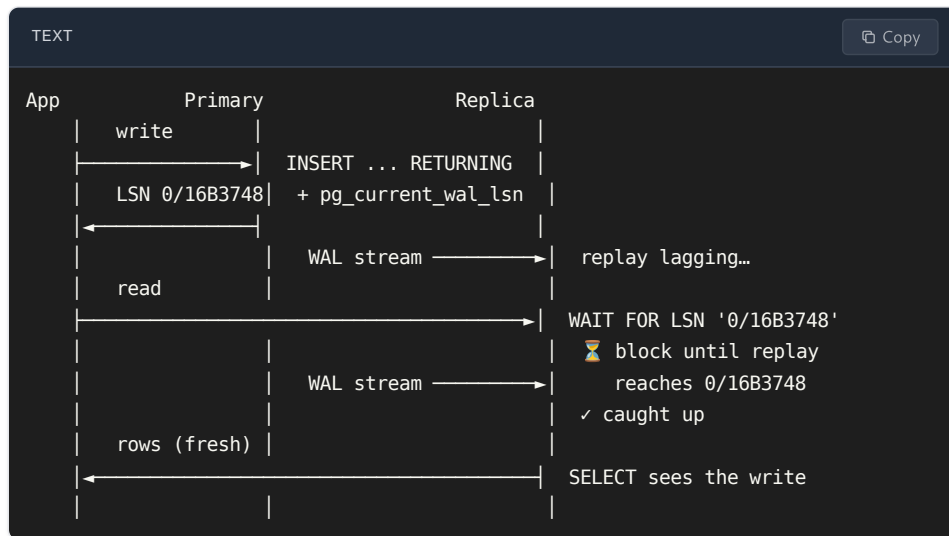
-- On the replica: block until replay reaches the LSN the write returned
WAIT FOR LSN '0/16B3748';

SELECT * FROM orders WHERE id = 42; -- guaranteed to see the write
  
```

You capture the LSN from the writing transaction on the primary and add it to the read path, and `WAIT FOR LSN` stalls just long enough for the replica to replay up to that point. No`

We use cookies for analytics. [Privacy](#)

more stale reads, no more sticky-session gymnastics.



Online Checksums

Data checksums can now be **enabled or disabled online**, without taking the cluster down or reinitializing it. Turning on checksums used to require a dump/reload or a downtime window with ``pg_checksums``; now it's a live operation.

Partition Management Lands In-Core

Declarative partitioning has been usable for years but missing obvious operations. PostgreSQL 19 fills two gaps:

```

SQL
-- Merge several partitions back into one

ALTER TABLE measurements

MERGE PARTITIONS (m_2024_01, m_2024_02, m_2024_03) INTO m_2024_q1;

-- Split one partition into several, in place

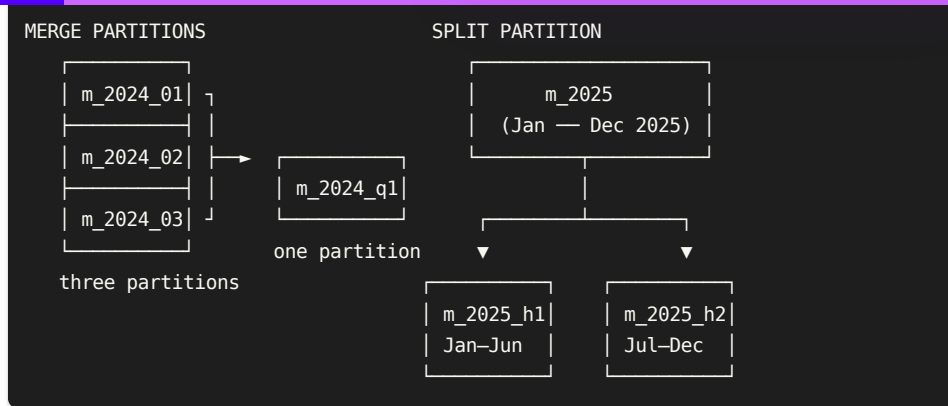
ALTER TABLE measurements

SPLIT PARTITION m_2025 INTO (

PARTITION m_2025_h1 FOR VALUES FROM ('2025-01-01') TO ('2025-07-01'),

PARTITION m_2025_h2 FOR VALUES FROM ('2025-07-01') TO ('2026-01-01')

);
  
```



Time-series and multi-tenant schemas that previously needed scripted detach/create/attach dances can now reorganize partitions with a single DDL statement.

SQL Features Developers Will Reach For

- `INSERT ... ON CONFLICT DO SELECT ... RETURNING` — finally return the conflicting row on an upsert collision, instead of forcing a no-op `DO UPDATE` just to get `RETURNING` to fire.
- `GROUP BY ALL` — group by every non-aggregate, non-window output column without retyping the list. A small quality-of-life win that analysts will love.
- `FOR PORTION OF` on `UPDATE` and `DELETE` — temporal/system-versioned tables get standard-compliant range-slicing of changes.
- **SQL/PGQ property graphs** — query relational data as a property graph using the SQL standard's graph syntax, no separate graph database required.
- **Richer jsonpath** — string functions (`lower()`, `upper()`, `initcap()`, `replace()`, `split_part()`, and the `trim()` family) now work inside jsonpath expressions.
- `random()` over ranges — now generates random `date` and `timestamp` values directly, handy for test-data generation.
- **COPY TO with native JSON output** — export query results straight to JSON without wrapping them in a function.
- **DDL-retrieval functions** — pull the `CREATE` statements for roles, tablespaces, and databases from SQL, which is a boon for backup tooling and IaC reconciliation.

Security and Authentication Changes

This is the section to read carefully before you upgrade, because two changes can break existing setups:

RADIUS AUTHENTICATION IS REMOVED

If any cluster's `pg_hba.conf` uses the `radius` auth method, it will not start on PostgreSQL 19. Migrate those entries to LDAP, certificate, or another supported method before testing the upgrade.

We use cookies for analytics. [Privacy](#)

The rest of the security story is additive and welcome:

- **SNI certificate routing** — a new server-side `pg_hosts.conf` lets a single PostgreSQL instance present different TLS certificates depending on the hostname the client requested (Server Name Indication). One server, many certs, proper multi-tenant TLS.
- **Password expiration warnings** — the new `password_expiration_warning_threshold` (default 7 days) warns users that their password is about to expire, instead of silently locking them out at the deadline.
- **md5 deprecation continues** — clients that authenticate with `md5` now get a warning after a successful login, governed by the `md5_password_warnings` setting. The writing is on the wall: move to SCRAM (`scram-sha-256`).

Observability: More Signals, Less Guessing

PostgreSQL 19 adds genuinely useful monitoring surface area:

- `pg_stat_lock` — per-lock-type statistics, so you can quantify which lock classes are actually contended instead of inferring it from `pg_locks` snapshots.
- `pg_stat_recovery` — detailed visibility into recovery operations on standbys.
- **Per-process `log_min_messages`** — set log verbosity per process type, so you can crank up logging on autovacuum or the walsender without drowning in chatter from everything else.
- **WAL full-page-write byte counts** now show up in `VACUUM` and `ANALYZE` log output, making it far easier to attribute WAL volume to maintenance work.
- `stats_reset` columns added across many statistics views, so you always know how stale a counter is.
- `pg_stat_progress_analyze` gains a `started_by` column, matching the new vacuum progress columns.

Default Changes That Will Bite the Unprepared

READ THIS BEFORE YOU BENCHMARK

- **JIT is now disabled by default.** For most OLTP workloads JIT cost more than it saved, so the default flipped to off. If you have analytical queries that genuinely benefited, re-enable `jit = on` explicitly and re-measure — don't assume your old query times carry over.
- `default_toast_compression` now defaults to `lz4` instead of `pglz`. New TOASTed values compress and decompress faster, but the binary differs — relevant if you have tooling that assumed pglz.
- `vacuumdb --analyze-only` now analyzes partitioned tables by default, which is almost certainly what you wanted, but it changes the runtime of existing maintenance scripts.

Query Federation: postgres-federated-query-partner

We use cookies for analytics. [Privacy](#)

For sharded and federated deployments, `postgres_fdw`` picks up two optimizations: array operations now push down to the remote server (less data dragged across the wire), and the planner can retrieve and use foreign-table statistics for better local planning of cross-server queries.

The Upgrade Checklist

If you want to be ready when PostgreSQL 19 GA ships, here's where to put your beta-testing effort:

1. **Audit `pg_hba.conf`` for `radius``** — it's removed; migrate those auth entries now.
2. **Audit for `md5`` auth** — plan the move to `scram-sha-256`` before the deprecation tightens.
3. **Re-benchmark with JIT off** — confirm whether any analytical workload needs `jit = on`` set back explicitly.
4. **Test parallel autovacuum and `REPACK CONCURRENTLY``** on a restore of your biggest, most-bloated table.
5. **Validate logical replication flows** — especially if you do logical-replication upgrades; sequence replication and online enablement change the runbook.
6. **Exercise extensions against 19** — every extension must be rebuilt for a new major version; confirm the ones you depend on have 19 builds.
7. **Test your monitoring** — the new `pg_stat_lock``, `pg_stat_recovery``, and progress-view columns are worth wiring into dashboards now.

BETA SCHEDULE

Beta 1 shipped June 4, 2026. Additional betas follow as needed, then one or more release candidates, with general availability targeted for September/October 2026. Grab the beta from the [official download page](#) and read the [full release notes](#) for the complete change list.

Should You Care Yet?

If you operate PostgreSQL at any scale, PostgreSQL 19 is the rare release where the headline features are the operational ones. Parallel autovacuum and `REPACK`` attack the two maintenance problems that wake DBAs up at 3 a.m. Online logical replication and `WAIT FOR LSN`` remove long-standing sharp edges from HA and migration work. And the default changes — JIT off, lz4 TOAST, RADIUS gone — mean a 19 upgrade is not a no-op; it deserves a real testing cycle.

The right move today is not to wait for GA in silence. Stand up a beta cluster, replay a realistic workload, and find out now whether the new default helps or hurts your specific

We use cookies for analytics. [Privacy](#)

system — so that when the September/October release lands, your upgrade is a formality instead of a fire drill.

Planning a PostgreSQL major-version upgrade and want it done with zero surprises? [Talk to the JusDB DBA team](#) — we run beta validation, build the upgrade runbook, and execute the cutover so you don't have to.

Share this article



0



0



JusDB Team

Official JusDB content team

Keep reading

POSTGRESQL

PostgreSQL Performance Tuning Playbook: ...

A repeatable, top-down method for tuning...

PostgreSQL

🕒 16 min

[Read →](#)

May 31, 2026

POSTGRESQL

PostgreSQL Architecture Deep Dive: Process...

Walk through PostgreSQL's multi-...

PostgreSQL

🕒 18 min

[Read →](#)

May 31, 2026

POSTGRESQL

PostgreSQL Explained (2026): Architecture,...

Comprehensive PostgreSQL guide...

PostgreSQL

🕒 5 min

[Read →](#)

May 13, 2026

Need Expert Help?

Need expert help? Our specialists can assist with these related services:



Database Performance Optimization

Comprehensive performance tuning

[Learn more →](#)



PostgreSQL Services

Patroni HA, query tuning, Oracle/MySQL migrations, 24/7 DBA - 1000+ PG inst

[Learn more](#)



PostgreSQL Migration

Oracle → PG, MySQL → PG, version upgrades 14→17, zero-downtime cutovers

We use cookies for analytics. [Privacy](#)



PostgreSQL Support

24/7 PostgreSQL support: VACUUM tuning, WAL/replication issues, 15-min critical-incident SLA

[Learn more →](#)



PostgreSQL High Availability

Patroni + streaming replication, 99.99% uptime, sub-30s automatic failover

[Learn more →](#)



PostgreSQL Cloud Migration

AWS RDS/Aurora, GCP Cloud SQL, Azure Flexible Server - on-prem to cloud with zero downtime

[Learn more →](#)

[Explore all our database services →](#)



JusDB

Uninterrupted database reliability engineered by experts. Trusted by startups to enterprises worldwide.



Databases

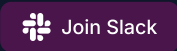
- MySQL
- PostgreSQL
- MongoDB
- Redis
- Cassandra
- SQL Server
- All Databases

Services

- Remote DBA
- Database SRE
- Performance Tuning
- Security Audits
- All Services

Community

Join database engineers sharing tips, troubleshooting, and best practices.



Contact

contact@jusdb.com

+91-9994791055



Trichy, Tamil Nadu, India



Whitefield, Bengaluru, Karnataka, India

POPULAR COMPARISONS

[MySQL vs PostgreSQL](#) [PostgreSQL vs MongoDB](#) [MySQL vs MariaDB](#) [MySQL vs MongoDB](#) [MySQL vs SQL Server](#) [PostgreSQL vs SQL Server](#) [Redis vs Valkey](#) [Redis vs Memcached](#) [MongoDB vs Cassandra](#) [MongoDB vs DynamoDB](#) [MongoDB vs Couchbase](#) [MongoDB vs Firestore](#) [DynamoDB vs Cosmos DB](#) [Cassandra vs ScyllaDB](#) [Elasticsearch vs OpenSearch](#) [PostgreSQL vs CockroachDB](#) [PostgreSQL vs YugabyteDB](#) [PostgreSQL vs Supabase](#) [CockroachDB vs YugabyteDB](#) [TimescaleDB vs InfluxDB](#) [ClickHouse vs Snowflake](#) [Druid vs Pinot](#) [Aurora vs RDS](#) [Oracle vs MySQL](#) [Oracle vs PostgreSQL](#) [BigQuery vs Snowflake](#) [Snowflake vs Redshift](#) [JusDB vs Percona](#) [JusDB vs Mydbsops](#) [JusDB vs Mafiree](#) [Best Remote DBA \(India\)](#) [All comparisons →](#)

© 2026 JusDB Technologies Pvt Ltd. All rights reserved.

[Privacy Policy](#) [Terms of Use](#) [Cookies Policy](#) [Security](#)

We use cookies for analytics. [Privacy](#)